

p my-runner-config --from-file=config.toml, you can update the runner config as illustrated below and apply it with `oc apply -f gitlab-runner.yml`.

```
yml
apiVersion: apps.gitlab.com/v1beta2
kind: Runner
metadata:
  name: gitlab-runner
spec:
  gitlabUrl: https://gitlab.com
  buildImage: alpine
  token: gitlab-runner-secret
  tags: openshift
  config: my-runner-config
```

1. The OpenShift runner should now appear green in your GitLab project settings CI/CD Settings > Runners.

SECTION: engineering/development/sec/secure/composition-analysis/_index.md

Composition Analysis

The Composition Analysis group at GitLab is charged with developing solutions which perform Container Scanning, and Software Composition Analysis. See the exhaustive list of projects the group maintains.

Common Links

- Slack channel: `gast-composition-analysis`
- Slack alias: `@securecompositionanalysisdev`
- Google groups: `composition-analysis-dev@gitlab.com`

How we work

Workflow

The Composition Analysis group largely follows GitLab's Engineering Workflow and Product Development Flow.

This includes:

- Issue triage
- Infradev triage
- Wider Community Merge Request Triage
- Retrospectives

Indicating Status and Raising Risk

We leverage the issue's health status feature to communicate the progress of the issue.

All issues should be marked On Track at the beginning of a milestone. This is done by the Epic DRI, or the Engineering Manager for unassigned, standalone issues.

Raising risk early is important. The more time we have, the more options we have. As such, the team reviews issues every week and discusses items that Need Attention or are At Risk to possibly course correct and re-assign resources based on the team's priorities.

Follow these steps when raising or downgrading risk:

1. Update the Health Status in the issue:
 1. On Track - the work will be completed within the planned milestone.
 1. Needs Attention - the issue is blocked or has other factors that need to be discussed.
 1. At Risk - the issue is in jeopardy of missing the cutoff to ship within the planned milestone.
1. Add a comment about why the risk has increased or decreased.
1. Copy the Engineering Manager and Product Manager in a comment.

Time-off Calendar

We use a shared calendar to see when team members are off work:

c629844cc273be17e067767febe12547bc40e129f26f0f17339030bff708cd0d5@group.calendar.google.com.

Access to this calendar is granted via this google group: sec-secure-composition-analysis@gitlab.com.

To share your time-off:

1. In Slack, find the Time Off by Deel application under the Apps menu.
1. Under Home, click on Your Events to show a dropdown.
1. Click on 'Calendar Sync' under the Settings break.
1. Click Add calendar under Additional calendars to include?. Use the calendar ID above.

To visualize the calendar:

1. In Google Calendar, find the Other calendars section in the left sidebar.
2. Click on the :heavylplussign: icon and then select Subscribe to calendar. Use the calendar ID above.

Reaction rotation

On top of our development roadmap, engineering teams need to perform tasks related to security vulnerabilities, support, maintenance, community contributions.

To avoid excessive context-switching, and better distribute the workload, our team reserves capacity for these tasks as part of milestone planning:

- Primary engineer. Fully allocated to the tasks below. They must prioritize these tasks above all other work, in the following order: Security, Support, Maintenance.
- Secondary engineer. Acts as a backup in case the primary engineer has an unplanned absence or exceeds their capacity. They must prioritize requests from the primary engineer, but otherwise focus on type::maintenance issues.

Neither engineer should be allocated to work on Features or critical deliverables. In the context of Cross-functional milestone planning, their allocation counts towards the bugs and maintenance ratio.

The rotation schedule follows the development cycle, which means using the start/end dates from the GitLab product milestones. When creating the schedule, the Engineering Manager should aim to minimize the number of back-to-back rotations that engineers do.

Please keep track of the actions you're doing during your rotation and add notes in the corresponding issue (e.g. copying tools command executed locally, sharing relevant changes to projects and processes, etc.),
You can use the Reaction Rotation issue template for this purpose.

At the end of the rotation, add the next engineers as Owners of @gitlab-org/secure/composition-analysis-dev/reaction-rotation, and remove the current engineers. This group is used to tag the engineers on rotation without having to look-up the schedule.

Responsibilities - Security

1. Triage vulnerabilities reported on the projects we maintain and help resolving them depending on their priority. (See Security vulnerabilities triaging process)
1. Check for SLA::Breached issues.
1. Check for security automation failures
1. Check for new security releases of our dependencies and ensure we use them:
 1. Upstream scanners (see Updating an upstream scanner)
 1. Container base images
 1. Application dependencies
 1. Programming language
1. Refine scheduled security issues.
1. Consider creating or updating any automation or tooling (related to security, maintainership or support!)

Responsibilities - Support

1. Monitor slack channels for questions, support requests, and alerts. While other team members may respond to these requests, the engineer assigned to the reaction rotation is expected to handle them primarily.

If a support engineer requests assistance via Slack and it requires investigation or debugging, they should be directed to raise an issue in the Request for Help project.

- gast-composition-analysis
- sapplication-security-testing
- sec-section
- sast-alerts
- fcontainerscanning

- gast-composition-analysis-alerts
- foperationalcontainerscanning
- sec-eng-requests-for-help

1. Monitor Section Sec Request For Help project for support requests.
1. Refine scheduled bugs and maintenance issues.

These items must be triaged continuously throughout the milestone which means they must be checked multiple times a week.

Responsibilities - Maintainership

1. Work with community contributors to help drive their merge requests to completion (more information on community contributions triaging process).
1. Run the License Alias Generator tool to update the list of known alias. You can find instructions here. This action should be performed once during a milestone.
1. Check for new versions of languages or package managers that we support, or deprecation / removal of support for the same and notify Engineering Manager and Product Manager via issue.
1. Check for new versions of our dependencies (not related to security):
 1. Upstream scanners (see Updating an upstream scanner).
 1. Container base images.
 1. Application dependencies.
 1. Programming language.
1. Check in on test failures. Check relevant slack channels (gast-composition-analysis-alerts, sast-alerts).
1. Check latest pipelines for any release failures. If any issue is preventing the automated release process from running, begin the release failure escalation process.
1. Consider creating or updating any automation or tooling (related to security, maintainership or support!).
1. Monitor failures and errors on license-db project, use the flicesedatabase Slack channel for communication about these items, so other team members can provide the support.
1. Monitor the Slack channel gast-composition-analysis-alerts for any incidents on the license-db infrastructure.
 - In case of an incident react with :eye: to indicate that you are looking into it.
 - If the incident isn't resolved in 30 minutes or more, investigate on it.
 - Write down in the insident Slack thread all the steps that were done to resolve it.
1. Monitor the Slack channels foperationalcontainerscanning and fcontainerscanning for alerts related to Operational Container Scanning (OCS) and Container Scanning.

Handover

1. As Reaction Rotation is continous process, post the handover status to the reaction rotation issue using the following template:

```
<details>
<summary>Reaction Rotation Handover Template</summary>
```

Reaction Rotation Handover Template

markdown

Ongoing Tasks and Issues

1. Security Tasks:

- Vulnerabilities:

- Issue 1234: Description of the vulnerability, current status, and next steps.

- Security Automation Failures:

- Automation Job 9012: Description of the failure, any initial troubleshooting steps taken, and next steps.

- Dependency Updates:

- Dependency Update 3456: Description, current status, and next steps for updating the dependency.

2. Support Tasks:

- Support Requests:

- Request 7890: Description of the support request in progress, current status, and any communication with the requester.

- Slack Channels Monitoring:

- gact-composition-analysis: Summary of recent discussions, any unresolved questions, and pending actions.

- sast-alerts: Summary of recent alerts, any unresolved issues, and pending actions.

3. Maintenance Tasks:

- Community Contributions:

- Merge Request 6789: Description of the contribution, current status, and next steps.

- Scheduled Bugs and Maintenance Issues:

- Issue 3456: Description of the issue, current status, and next steps.

Additional Notes

1. Any additional context or notes relevant to the next engineer:

- Additional Note.

This handover template ensures that the incoming engineer is fully informed of the current status of tasks, issues, and any important context. It facilitates a smooth transition and minimizes the risk of overlooking critical responsibilities.

</details>

1. Assign ownership of the Reaction Rotation group to the new engineers.

Security vulnerabilities triaging process

We are responsible for triaging vulnerabilities reported on 2 sets of projects: the projects maintained by GitLab and the upstream scanner software we might depend on. Though, we have different processes that apply depending on the situation.

See the Application Security Testing sub-department vulnerability management process.

Security Policy

We prioritize findings by their CVSS severities and SLAs. Start with Critical and High but also look for issues that are connected to vulnerabilities and have an SLA::Near Breach label. These vulnerabilities might have a lower CVSS score but letting them reach SLA breach will count as 'past due' security issues and affect FedRAMP compliance.

Please utilize all the time you have set aside. If you complete all the ones at Critical and High, please continue to triage - we want to address all findings but we are working in a risk based order.

SLA::Breached issues

Sometimes we might have SLA::Breached issues that need to be handled ASAP. You can view the number of those issues in the Tableau dashboard. SLA::Breached issue may appear for many reasons including:

- A medium or low vulnerability that is not handled because it never got priority. Please notice that a low vulnerability might lead to a severity::1 issue since it might get its score from different sources.
- Issues that are never closed even if the relevant vulnerability is resolved or dismissed.

You can search for SLA::Breached issues in the issue tracker using the following label filters:

- Severity 1
- Severity 2
- Severity 3
- Severity 4

Triaging vulnerabilities

We use the Vulnerability Report with filters to focus on items matching our policy and reported on the relevant projects.

1. Analyzers Vulnerability Report

- To configure the report manually, select all shared, container scanning, and dependency scanning projects and apply the Still detected activity filter and apply the Needs Triage status.

1. License-db Vulnerability Report

- To configure the report manually, select all license-db projects and apply the Still detected activity filter and apply the Needs Triage status.

For each item, investigate and either dismiss or confirm it. If it's not clear whether there's indeed a threat, escalate to our Application Security team.

> Refer to Vulnerability status definitions in case you are unsure of what each of them mean.

Upstream scanners vulnerabilities

This only applies to projects NOT maintained by GitLab.

We review vulnerabilities detected on upstream scanners when upgrading to a newer version. See the Security checks when updating an upstream scanner section.

We currently have a limited capacity to triage vulnerabilities reported on our upstream scanners. Continuously triaging vulnerabilities reported for these projects is done on a best effort basis.

Triaging vulnerabilities

We use the Vulnerability Report with filters to focus on items matching our policy and reported on the relevant projects.

1. Upstream Scanners Vulnerability Report

- To configure the report manually, select all upstream scanner projects.

For vulnerabilities discovered in upstream scanners, an issue must be created in GitLab's issue tracker, and we should work with the relevant Open Source community to help provide a resolution. As a last resort, we can patch locally or fork the upstream project temporarily to fix the vulnerability sooner.

Dismissing a vulnerability

When there is no doubt a vulnerability is a false-positive, it can be "Dismissed" unless it related to a FedRAMP image (fips).

Select the "Dismiss" option from the vulnerability status options.

Finally, make sure to comment on the vulnerability status change notification to explain why.

Low risk findings that can be dismissed

Because of both the way severity is generically set in CVSS and automated scanners do not have all context for an application, many findings which may be high risk in other environments or scenarios are low risk for our users. The containers ingest code from a user project and that user has developer access, and the containers are ephemeral and related to a specific pipeline.

In some other cases, a finding is related to an upstream dependency or Operating System and there is no fix available and no fix planned. Please be sure to mark this issues using the labels; blocked or blocked upstream.

When an issue is both blocked for a few releases and low risk you may dismiss the finding with a note as to the reasoning. If there is an open issue notify the Application Security team with your specific reasoning and close the issue (if applicable). In the future we will specifically want to tag everything related to these findings as won't fix or blocked when they are being closed, for now that is only available on issues and not findings.

The following class of container scan vulnerabilities can be considered low risk:

- Many kernel-related findings will be at a decrease of risk and hence severity because of the way our process works with temporary containers with limited inputs which are developer-

controlled.

- Issues related to a software stack that will not apply to the analyzer e.g GUI related issues, issues in Bluetooth drivers, browser-related issues which require browser running in non-headless mode, etc.
- S3 or S4 findings with complex exploit method or limited risk which have no fix available, or the upstream has stated there are no plans to release a patch.
- Denial of Service (of the container/analyzer) as these containers run in ephemeral pipelines, are automatically stopped once a timeout is reached, and are accepting in code from users who already have developer access. This as a result is not an expansion of the risk profile.
- Random number generator issues (where the numbers are not random) as we don't use random numbers for security purposes from the containers. (At the time this was last updated these were true, please use your knowledge of our analyzers or ask if unsure)"

To add items to the list above discuss repeatable finding patterns with Application Security, get approval from a leader in the security section, and add to this list.

Confirming a vulnerability

If the vulnerability impacts a dependency:

1. Evaluate if the dependency (software library, system library, base image, etc.) can be upgraded or removed.

1. Set the vulnerability status to "Confirmed".

1. Release a new version of the analyzer with the dependency upgrade/removal and follow the process on resolving a vulnerability.

For all other confirmed vulnerabilities, create a security issue to discuss and track the remediation.

Resolving a vulnerability

When a vulnerability has been remediated, it can be "Resolved". When doing so, comment how it was

remediated, then select the "Resolve" option from the vulnerability status options, and close the related vulnerability issue.

Creating security issues

Unfortunately, creating a security issue can't be done yet via the "create issue" button from the vulnerability page or security dashboard as this only works when creating an issue in the same project where the error was reported and we've disabled the embedded issue tracker in our projects.

Instead, in our workflow we open all our issues in the main GitLab project.

As a workaround, you can copy and paste the content of the vulnerability page (this keeps markdown formatting!). Please also follow our Security guidelines about creating new security issues.

You can leverage quick actions to add the necessary labels.

text

/confidential

/label ~security ~"type::bug" ~"bug::vulnerability"

/label ~"section::sec" ~"devops::application security testing" ~"group::composition analysis"

<!-- depending on the affected project: -->

/label ~"Category:Software Composition Analysis"

/label ~"Category:Container Scanning"

It's important to add the ~security and ~"bug::vulnerability" labels as described above, because the AppSec Escalation Engine will automatically pick up any issues with these labels and add additional labels ~security-sp-label-missing and ~security-triage-appsec as well as mention the issue in the sec-appsec Slack channel. At this point, the Stable Counterpart or Application Security team triage person will pick up the issue and assign a severity as part of the appsec triage rotation.

Once the issue is created, please add it to the vulnerability's linked items for ease of tracking.

Developers reporting the security issue should help the Application Security team assess the impact of the vulnerability, and update the issue description with an Impact section.

If immediate feedback is required, then add a comment to the vulnerability issue with an @-mention directed at one of the Security Engineers listed in the Stable Counterpart section, or ping them on slack.

Release failure process

If the image release process is failing, an incident should be created to track how it was detected, escalated, and resolved. Documenting our incidents makes it possible to search for previous incidents by keyword, labels, and other issue filters. We open all of our incidents in the main GitLab project.

1. Open a new incident

and add a description of the problem along with any reproduction steps. Add the following labels so that we can track the incidents that have impacted composition analysis in the future.

text

<!--

Select one of the following severities

Ref: <https://handbook.gitlab.com/handbook/engineering/infrastructure/engineering-productivity/issue-triage/severity>

-->

/label ~"severity::1"

/severity S1

```
/label ~"severity::2"  
/severity S2
```

```
/label ~"severity::3"  
/severity S3
```

```
/label ~"severity::4"  
/severity S4
```

```
<!--
```

Select one of the following priorities

Ref: <https://handbook.gitlab.com/handbook/engineering/infrastructure/engineering-productivity/issue-triage/priority>

```
-->
```

```
/label ~"priority::1"  
/label ~"priority::2"  
/label ~"priority::3"  
/label ~"priority::4"
```

```
/label ~"section::sec" ~"devops::application security testing" ~"group::composition  
analysis" ~"type::bug" ~"bug::availability"
```

```
<!--
```

Select one of the following categories

```
-->
```

```
/label ~"Category:Dependency Scanning"  
/label ~"Category:Container Scanning"  
/label ~"Category:License Compliance"
```

1. Assign the incident to the engineer currently on the maintainership reaction rotation.
1. Link any related issues or zoom meetings with the quick actions to record incident timeline events. Ensure that an event exists for the incident start, detection, resolution, and any other events that you feel are worth highlighting as part of the incident response.
1. Upon fixing the issue, include a detailed summary of the resolution and any initial follow up actions that should be completed. Lastly, an entry for incident should be added to the weekly composition analysis group meeting so that it may be reviewed with the entire group.

Example Incident(s)

- PHP Composer segfaults in gemnasium analyzer

Maintenance triaging process

To help our Engineering Manager prioritize maintenance issues, the engineering team assigns them a priority label.

1. Leverage the Maintenance issues board.
1. For each open issue that has no Priority label ("Open" column), shortly investigate the

issue (< 1h) and comment with your findings. Make sure the correct sub-category label is applied per our Work type classification (e.g. ~maintenance::refactor).

Code review

Upon joining Composition Analysis group, team members are suppose to become either reviewers or maintainers for all projects maintained by the group. The process how to become maintainer is described in the general Code review guidelines.

Projects

The Composition Analysis group maintains several projects to provide our scanning capabilities.

Shared

- common library

Dependency Scanning

- gemnasium analyzer
- gemnasium-maven-plugin
- gemnasium-gradle-plugin
- dependency-scanning analyzer
- Internal only gitlab-depscan

Additional notes:

- gemnasium-db is maintained by the Vulnerability Research group.

Container Scanning

- container-scanning analyzer

License-db

- advisory-processor
- deployment
- license-exporter
- license-feeder
- license-interfacer
- license-processor
- schema
- PMDB tools
- Static Reachability Modules Scraper

Operational Container Scanning

- trivy-k8s-wrapper
- OCS module
- Cluster Image Scanning related code

- Needed for Operational Container Scanning feature.

The OCS module is part of the gitlab-agent project which is maintained by the Environments group. The Composition Analysis group is responsible for maintaining only the OCS module.

Semver dialects gem

- semverdialects

CI/CD components

- Dependency Scanning
- Container Scanning

Upstream scanner mirrors

As some of our analyzers rely on open source software, we include them in our security testing to increase coverage and reduce risk.

To do so, we mirror their repository and execute our security scans on them (when relevant):

- trivy
- trivy-db
- trivy-java-db
 - The oras-mirror-config

branch contains a scheduled pipeline that keeps the registry.gitlab.com/gitlab-org/security-products/dependencies/trivy-java-db:1 image up to date.

- trivy-db-data
- trivy-db-glad
- vuln-list-update

The vulnerabilities reported on the currently used version of the scanner are automatically reported in the group level Vulnerability Report and triaged as part of our security vulnerabilities triaging process.

Setting up a mirror

1. create a new project in <https://gitlab.com/gitlab-org/security-products/dependencies> (blank project).
2. set up the project repository as a pull mirror of the upstream repository.
3. find the git tag that matches the version currently used by our analyzer (usually represented by the SCANNERVERSION variable in the analyzer's Dockerfile). Use exact commit if there is no git tag for the corresponding release we use.
4. create a branch from that ref following naming convention VERSION-security-checks where VERSION is the version of the upstream scanner we currently use (e.g. v6.12.0).
5. add a .gitlab-ci.yml configuration file to configure all compatible security scans.
6. make VERSION-security-checks the default branch, so that reported vulnerabilities are showing up on the dashboards and vulnerability reports.

Updating an upstream scanner

We check for new releases of the upstream scanners on a monthly basis, as part of our release issue. When an update is available, a new issue is created using the update scanner issue template and added to the next milestone.

Upstream Tools and Analyzers list

- 1. Trivy.
 - 1. Container Scanning
 - 1. Trivy K8S wrapper
- 1. OpenTofu
 - 1. Deployment
- 1. OpenTofu component
 - 1. Deployment

Every analyzer relying on an upstream scanner has a "How to update the upstream Scanner" section in their readme detailing the process. This includes a verification for possible new security vulnerabilities and a license check which are detailed below.

Security checks when updating an upstream scanner

Before releasing an analyzer with a newer version of its upstream scanner, we must ensure it is exempt of security vulnerabilities matching our current policy.

- 1. checkout the new tag (or commit) and create a new branch from it following naming convention NEWVERSION-security-checks.
- 1. copy/paste the existing .gitlab-ci.yml configuration file from the current VERSION-security-check branch.
- 1. if there are new findings matching our policy, address them according to our triage process.
- 1. only when above mentioned findings are fixed, update the defaultbranch to be NEWVERSION-security-checks and proceed with the update of the analyzer to use this newer version.

License check when updating an upstream scanner

Before releasing an analyzer with a newer version of its upstream scanner, we must ensure its license has not changed or is still compatible with our policy.

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://10az.online.tableau.com//site/gitlab/views/TopEngineeringMetrics16989570521080/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="composition analysis" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px"
src="https://10az.online.tableau.com//site/gitlab/views/MRMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="composition analysis" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/DRAFTFlakyste
stissues/MonthlyFlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="composition analysis" >}}
```

{{< /tableau >}}

{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}

{{< tableau/filters "GROUPLABEL"="composition analysis" >}}

{{< /tableau >}}

Monitoring

- Stage Group dashboad on Grafana
- Continuous Vulnerability Scanning (background processing on the gitlab.com rails platform)

Runbooks

See our Composition Analysis Runbooks page.

Slack alerts

The following table presents the various Gitlab projects that can generate slack alerts. Please notice that this doesn't include alerts generated by Sentry.

Project	Target slack channel	Additional Info
Gemnasium	gast-composition-analysis-alerts	
Deployment	gast-composition-analysis-alerts	Multiple alerts can be generated mainly from scheduled pipelines for exporting or feeding license/advisory/epss/kev data
Container Scanning	fcontainerscanning	Failures on default branch
trivy-k8s-wrapper	foperationalcontainerscanning	Failures on default branch
trivy-db-glad	gast-composition-analysis-alerts	Failures on default branch
Dependency Scanning Analyzer	gast-composition-analysis-alerts	Failures on default branch
Dependency Scanning Component	gast-composition-analysis-alerts	Failures on default branch

SECTION: engineering/development/sec/secure/composition-analysis/runbooks/_index.md

Overview

This page lists runbooks used by the Composition Analysis team.

Runbooks

Package Metadata DB runbook

Enable/Disable SemverDialect Errors in Sentry

SECTION: engineering/development/sec/secure/composition-analysis/runbooks/semver-dialect-errors-in-sentry.md

Overview

This runbook provides instructions for enabling the reporting into Sentry of SemverDialect errors generated within the GitLab rails application. This reporting is behind the `tracksemverdialecterrorsforcvsinsentry` OPS feature flag, which is disabled by default. This prevents flooding our Sentry system with too many errors generated from processing invalid or unsupported user data on gitlab.com.

Note: this feature flag should be removed by 18.4: <https://gitlab.com/gitlab-org/gitlab/-/issues/491612>

Procedure

1. Enable the feature flag on production environment: `/chatops run feature set tracksemverdialecterrorsforcvsinsentry true --production`
2. Monitor SemverDialects errors on Sentry: <https://new-sentry.gitlab.net/organizations/gitlab/issues/?query=is%3Aunresolved+SemverDialects>
3. Create issues for reported errors that need to be addressed.
4. Disable the feature flag on production environment: `/chatops run feature set tracksemverdialecterrorsforcvsinsentry false --production`
5. Fix the issue.
6. Repeat

SECTION: engineering/development/sec/secure/dynamic-analysis/api-security.md

API Security

The API Security team is a standalone team which is part of the Dynamic Analysis group at GitLab. It is charged with developing solutions which perform Fuzzing.

Repo Purpose
---- -
API Fuzzer - Private GitLab's API Fuzzing scanner.

Important Fuzzing repositories

Repo Purpose
---- -
API Security Private - The API Security tool performs API Fuzzing and API DAST scans
API Fuzzing E2E Tests Private - API End to End Tests

DAST API demos	Public - DAST API demos linked from the documentation.
API Fuzzing demos	Public - API Fuzzing demos linked from the documentation.
API Fuzzing demos	Public - API Fuzzing demos linked from the documentation (har/openapi branches).

How to Contact Us

- Slack channel: [gast-dynamic-analysis](#)
- Slack alias: [@securedynamicanalysisbe](#)
- Google groups: [dynamic-analysis-be@gitlab.com](#)
- GitLab mention: [@gitlab-org/secure/dynamic-analysis-be](#)

How We Work

The Dynamic Analysis group largely follows GitLab's Product Development Flow.

Issues worked by this team are backend-centric and are typically in one the above repos, vendored templates, and GitLab's Rails monolith. At times, issues can require support from AST's frontend team if UI changes are required. We will require more notice for initiatives like these.

Repeated tasks

There are several maintenance tasks that need to be completed each milestone. Each iteration, an issue is opened and assigned to an engineer on a rotating basis. Those rotating tasks are:

- Review upstream changes, and open an issue to upgrade DAST if the upstream changes provide important improvements
- Review the security dashboard for DAST and address all critical and high issues. Review the dashboards for upstream projects, ZAP and ZAP Extensions

Fuzzing Technologies

- The API Security product is built using mostly C with some small amounts of Python. Our engineers use Windows VMs for development.

Specialized Labels

When opening up issues, the following label snippet often added:

```
text
/label ~"Category:API Security"
/label ~"group::dynamic analysis"
/label ~"devops::application security testing"
/label ~"backend"
/label ~"section::sec"
```

Dashboards


```
{{< tableau height="600px" toolbar="hidden" src="https://10az.online.tableau.com//site/gitlab/v  
iews/TopEngineeringMetrics16989570521080/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="dynamic analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px"  
src="https://10az.online.tableau.com//site/gitlab/views/MRMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="dynamic analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/DRAFTFlakyte  
stissues/MonthlyFlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="dynamic analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/SlowRSpecTes  
tsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="dynamic analysis" >}}  
{{< /tableau >}}
```

Targets

For our Merge Request types, we have an initial soft target ratio of 60% features, 30% maintenance, and 10% bugs based on the cross-functional prioritization process. This is not a hard target and we expect to see variation in this ratio as we mature and our focus evolves.

Support Requests

The Dynamic Analysis engineering team provides support to GitLab Support Engineers following the process outlined in the Sec Section support project.

Issue Boards

- API Security - Delivery Workflow Board
- API Security Planning Board
- Dynamic Analysis EM Board

SECTION: engineering/development/sec/secure/dynamic-analysis/dynamic-analysis.md

Dynamic Analysis

The Dynamic Analysis group at GitLab is charged with developing solutions which perform Dynamic Analysis Software Testing (DAST) and Fuzzing. Our work is a mix of open and closed source code.

Mission

To support the success of GitLab by developing highly usable, high quality tools for customers to build more secure software. The Dynamic Analysis group at GitLab is charged with developing

solutions which perform API Security Testing, Dynamic Analysis Software Testing (DAST) and Fuzzing.

Top Priorities (FY25)

Theme: Increase adoption

- Unify DAST offerings - Reduce confusion by removing proxy-based DAST and maturing browser-based DAST by adding active checks and improving performance
- API Discovery - API discovery will analyze applications to produce an OpenAPI document describing the web APIs expected behavior. This schema document is then used by API Security testing (formerly called DAST API) to perform security scans. Automating this will close the gap for customers whose applications include web APIs that haven't been documented and aren't being tested today.
- API Security Checks refresh - Comprehensively review and update API Security Testing checks (rules). By ensuring we address both the OWASP API Security Top 10:2023 and the OWASP Top 10:2021 (web applications), our customers will be confident in our ability to detect critical risks.
- DAST Performance - Deduplicate scanning similar pages to reduce DAST scan times without reducing security coverage. Long scan times are a major barrier that discourages customers from incorporating DAST scans into DevSecOps processes.

Customer outcomes We are Driving

The Dynamic Analysis team builds features for our API Security, DAST, and Fuzzing categories. These features enable our Ultimate customers to incorporate security testing earlier in the development lifecycle for their web applications and APIs. By identifying vulnerabilities earlier in the SDLC, we enable customers to more efficiently reduce the security risks in their web apps and APIs.

Our features identify vulnerabilities in different ways than GitLab's other security analyzers do, and each tool should be used in tandem with other security analyzers such as SAST, SCA (dependency scanning), and secret detection to provide full coverage.

API Security, DAST, and Fuzzing are Ultimate features. Increasing adoption of these features helps improve customer retention, and helps drive revenue for GitLab Ultimate.

Important DAST Repositories

Repo	Purpose
DAST	The DAST Analyzer which is deployed as a docker image.
Browserker	Private - GitLab's DAST browser-based analyzer.
DAST CWE Checks	Private - DAST browser-based analyzer vulnerability definitions.
DAST Chromium	Private - DAST browser-based analyzer dependencies, such as the Chromium browser.

Important Fuzzing repositories

Repo	Purpose
----	-----
API Security	Private - The API Security tool performs API Fuzzing and API DAST scans.
GitLab Protocol Fuzzer (CE)	GitLab's Protocol Fuzzer (Community Edition), previously Peach Protocol Fuzzer.
GitLab Protocol Fuzzer (EE)	Private - GitLab's Protocol Fuzzer (Enterprise Edition), previously Peach Protocol Fuzzer, includes licensed components.
API Fuzzing E2E Tests	Private - API End to End Tests.
GitLab Cov Fuzz	Private - GitLab's coverage fuzzing orchestration that integrates fuzzing engines/results with GitLab CI and GitLab Security Dashboard.
HAR Recorder	A utility to record HAR files based on web traffic.

Open Source Fuzzers

Repo	Purpose
----	-----
JSfuzz	Javascript Fuzzer.
Pythonfuzz	Python Fuzzer.
Javafuzz	Java Fuzzer.
Coverage Fuzzing Examples	Coverage fuzzing examples in 7+ languages/Fuzzers.

How to Contact Us

- Slack channel: [gast-dynamic-analysis](#), [fast-api-security](#), [fast-fuzz-testing](#)
- Slack alias: [@securedynamicanalysisbe](#)
- Google groups: [dynamic-analysis-be@gitlab.com](#)
- GitLab mention: [@gitlab-org/secure/dynamic-analysis-be](#)

Support Requests

The Dynamic Analysis engineering team provides support to GitLab Support Engineers following the process outlined in the [Sec Section](#) support project.

Other Contact

The DAST team also monitors [sapplication-security-testing](#) and [sec-section](#). Both these channels are for wider AST topics, however are a good place to start if you are not sure which group in AST to contact.

How We Work

mermaid
timeline

title Annual Kanban Ceremonies
section Q1
 OKR Development Sync
M1
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync

: W3
 Weekly Team Sync
: W4
 Weekly Team Sync
M2
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

M3
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

section Q2
 OKR Development Sync

M4
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

M5
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

M6
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

section Q3
 OKR Development Sync

M7
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

M8
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

M9
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

section Q4
 OKR Development Sync

M10
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

M11
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync
M12
 Milestone Planning Sync
: W1
 Weekly Team Sync
: W2
 Weekly Team Sync
: W3
 Weekly Team Sync
: W4
 Weekly Team Sync

The Dynamic Analysis group works according to Kanban principles, with the addition of ceremonies quarterly for OKR development, monthly for release planning, and weekly for syncing on epic and issue reacting, triaging, refinement, impediment resolution. By having the team operate and think at each of these cadences, it can effectively set high-level hierarchical goals, ensure product priorities are planned, and prevent epics and issue work from stalling or falling through the cracks.

Hallway Monitor Bot

For all quarterly, monthly, and weekly Kanban ceremonies, the Hallway Monitor Bot is our guiding star, allowing the team to pontificate and plan at all cadences, ensuring we get ahead of future work. The bot will schedule quarterly OKR review meetings, monthly release planning and retro meetings, and will update or weekly sync guide with the latest team member assigned to the temporary reaction rotation role. We should strive to automate as many of our business process as possible, and the Hallway Monitor Bot Configuration and Templates allow for many of your automation ideas. Please help contribute to the bot to help keep our team as efficient as possible.

Quarterly OKR Development

mermaid
timeline

section Q1
 OKR Development Sync
section Q2
 OKR Development Sync
section Q3
 OKR Development Sync
section Q4
 OKR Development Sync

Four weeks before the end of a fiscal quarter, our bot will automatically create an OKR planning issue for the group and schedule a calendar event.

The group will get together two weeks before the next quarter begins to create and review all OKRs for which the team will ultimately be responsible. This includes reviewing all ancestor OKRs for the stage and up, creating or updating any new or existing OKRs with additional details, and assigning DRIs to each OKR to assist with health updates.

Important Links

- List of Team Objectives & Key Results
- List of Stage Objectives & Key Results (Coming Soon)
- List of CEO Objectives & Key Results (Coming Soon)

Quarterly Outputs

- Polished OKRs for the group with assigned DRIs
- Links to any associated epics or issues

Milestone Release Planning

mermaid
 timeline

section M1
 Milestone Planning Sync
 section M2
 Milestone Planning Sync
 section M3
 Milestone Planning Sync
 section M4
 Milestone Planning Sync
 section M5
 Milestone Planning Sync
 section M6
 Milestone Planning Sync
 section M7
 Milestone Planning Sync
 section M8
 Milestone Planning Sync
 section M9
 Milestone Planning Sync
 section M10
 Milestone Planning Sync
 section M11
 Milestone Planning Sync
 section M12
 Milestone Planning Sync

Two weeks before a release milestone is kicked off, our Hallway Monitor Bot will automatically create a monthly release planning issue for the group and schedule a calendar event. This issue serves as the Kanban backlog for the milestone, aligning PM priorities with the EM and all of the ICs. The issue is divided into three sections:

- Work to Release - all work that we are committing to release during this milestone
- Work to Start/Continue - new work being brought in or existing work rolling over from a previous milestone
- Work to Blueprint - work that needs high-level planning and breakdown this milestone before development work in the next milestone

On the date of a release milestone kick-off, we will review the list of Dynamic Analysis Group direction priorities with product and ensure all of the priorities for the current milestone are reflected in the monthly release planning issue in the "Work to Release" and "Work to Start/Continue" sections. Medium to large sized bodies of work identified for the next milestone by our PM should be added to the "Work to Blueprint" section for high-level blueprinting and breakdown of the work for pick-up in the next milestone.

Product sets all of our high-level priorities, and in addition to the above, another output from this kick-off meeting can be an MR update to direction page with an updated mapping of each priority to a group issue/epic and/or targeted milestone. This process allows for the EM and ICs to give direct feedback to our PM for how we'd like to organize the work and which milestone we think we can get to future roadmap items.

All product priority-based issues will exist alongside our comprehensive list of all Dynamic Analysis Group issues. All product priority-based epics will exist alongside our comprehensive list of all Dynamic Analysis Group epics.

To complete this mapping, product priority-based issues or epics should be created in gitlab-org/gitlab or gitlab-org (respectively) with the same title of the priority and the following labels. The type::feature label is what distinguishes a product priority from other group epic work.

text

/label ~"section::sec"

/label ~"devops::application security testing"

/label ~"group::dynamic analysis"

/label ~"type::feature"

Milestone Retrospectives

After the completion of any milestone, the planning issue for that milestone is used to capture team retros in the comments. Before the next milestone planning kickoff meeting, each team member should answer the following questions in a comment on that issue:

- · what went well this release?
- · what didn't go well this release?
- · what can we improve going forward?
- · what praise do you have for the group?

Important Links

- Dynamic Analysis Group Direction Priorities
- List of all Dynamic Analysis Milestone Planning Issues & Retros

Milestone Outputs

- MR to update the group direction with all priorities and targeted releases
 - Links to polished epics or issues for each product priority
 - Flush out as many child epics and issues tagged to milestones as possible (or a single blueprinting issue)
- Team retrospective comments will be reviewed inside of the previous milestone's planning issue

Weekly Huddles & Reaction Coordination

Monday's Developers Huddle

Every Monday, a calendar event with an attached agenda exists for a synchronous development group meeting. This meeting is engineering focused and includes all engineers on the team and the engineering manager. The purpose of this meeting is to discuss development efforts, strategies, tooling, active MRs, and to guide each other's coding and review efforts. Additionally, the team will discuss all of the active blueprinting items defined in the current

milestone planning issue, using this time to discuss implementation plans, spikes, and architectural decisions; all in hope of establishing epics and issues for work in the next milestone.

Tuesday's General Huddle

Every Tuesday, a calendar event with an attached agenda exists for a synchronous general group meeting. This meeting is product focused and includes all engineers on the team, the engineering manager, and the product manager. The purpose of this meeting is to discuss product efforts, timelines, and serves as a direct two-way feedback mechanism between product and the engineering team. We will review the current milestone planning issue as our transient backlog and ensure that we are staying focused on these priorities. The group will also rotate to a new Reaction Coordinator, discuss any unresolved and significant/product impediments blocking the team, and host general discussions valuable to a synchronous team meetup with our product manager.

Reaction rotation

On top of our development roadmap, engineering teams need to perform tasks related to vulnerability management, support, maintenance, community contributions.

The rotation schedule follows the development cycle, which means using the start/end dates from the GitLab product milestones. When creating the schedule, the Engineering Manager should aim to minimize the number of back-to-back rotations that engineers do.

Request For Help (RFH) Resolution Guide

This guide outlines the standard procedure for handling and resolving Request For Help (RFH) issues. Following these guidelines ensures consistent customer service and proper issue management.

Engineers participating in reaction coordination must make sure their GitLab handles are included in the RFH template so they are notified when a new RFH is opened.

When a new RFH is opened the reaction coordinator will be assigned and starts the investigation. If the reaction coordinator rolls off reaction coordination before the RFH is completed, the RFH should be handed off to an incoming reaction coordinator.

At least once, each milestone, the Reaction Coordinator is responsible for triaging existing RFH issues.

1. Has an engineer engaged with the RFH? If not, assign RFH to one of the reaction coordination engineers to work on.
1. Is the RFH a candidate to be closed? If so, close the RFH issue with any needed notes.
1. Check zendesk for latest customer response.

When to Close an RFH

RFH issues can take a long time to resolve with numerous back and forth communications between engineering, support, and the customer. Sometimes customers will stop responding if they get

past the issue, a workaround works, or they decide to stop investing in the support case. To limit the amount of issue maintenance that is required, RFH issues should be closed when there are no more steps to be performed or when the last step is a confirmation from the customer. When the last step is a confirmation from the customer close the issue with a message similar to:

Closing this issue out as a fix/workaround has been provided. Please feel free to re-open this issue if provided solution doesn't work or the customer has additional questions/concerns.

This prevents engineers from having to circle back and close out issues if the customer never responds.

It also helps keep our KPIs looking good (how many RFHs are open, how long to resolve).

An RFH issue can be closed under the following circumstances:

1. Confirmed resolution

- The customer has confirmed the issue has been resolved

1. High confidence workaround or resolution

- Engineer provides a high confidence workaround or resolution
- Engineer closes issue with a note to reopen if the workaround or resolution doesn't work

1. Feature request backlog

- The RFH is for a feature that cannot be prioritized immediately
- An issue has been created in the backlog
- The backlog issue has been linked to the original RFH
- A note has been added to the RFH pointing to the linked issue
- Engineer closes issue

1. Immediate Implementation

- The issue has been worked on immediately
- Changes have been merged and are ready for customer testing
- Engineer closes issue with a note to reopen if the workaround or resolution doesn't work

1. No Customer Response

- The RFH has received a response from support
- There has been no customer reply for a prolonged period (15 days)
- Engineer closes issue with a note to reopen if the customer responds

Best Practices

- Acknowledge receipt of new RFH issues promptly
- Set clear expectations about resolution timeframes
- Link related issues or documentation when applicable
- Provide detailed explanations when closing issues to ensure knowledge transfer

Issue Status Monitoring

Regular review of open RFH issues should be conducted to ensure no issues remain unaddressed for extended periods.

Vulnerability Management

1. Triage vulnerabilities reported on the projects we maintain and help resolving them depending on their priority. (See Security vulnerabilities triaging process)
1. Check for SLA::Breached issues.
1. Check for security automation failures
1. Check for new security releases of our dependencies and ensure we use them:
 1. Upstream scanners (see Updating an upstream scanner)
 1. Container base images
 1. Application dependencies
 1. Programming language
1. Refine scheduled security issues.
1. Consider creating or updating any automation or tooling (related to security, maintainership or support!)

Security vulnerabilities triaging process

We are responsible for triaging vulnerabilities reported on 2 sets of projects: the projects maintained by GitLab and the upstream scanner software we might depend on. Though, we have different processes that apply depending on the situation.

See the Application Security Testing sub-department vulnerability management process.

Security Policy

We prioritize findings by their CVSS severities and SLAs. Start with Critical and High but also look for issues that are connected to vulnerabilities and have an SLA::Near Breach label. These vulnerabilities might have a lower CVSS score but letting them reach SLA breach will count as 'past due' security issues and affect FedRAMP compliance.

Ensure security issues (for FedRAMP compliance or platform security) are created or updated, either manually or through automation. Follow up with creation of Deviation Requests if necessary.

Please utilize all the time you have set aside. If you complete all the ones at Critical and High, please continue to triage - we want to address all findings but we are working in a risk based order.

SLA::Breached issues

Sometimes we might have SLA::Breached issues that need to be handled ASAP. You can view the number of those issues in the Tableau dashboard (Note: This does not require SAFE access). SLA::Breached issue may appear for many reasons including:

- A medium or low vulnerability that is not handled because it never got priority. Please notice that a low vulnerability might lead to a severity::1 issue since it might get its score from different sources.
- Issues that are never closed even if the relevant vulnerability is resolved or dismissed.

If SLAs are breached, and if an SLA exception request is not already created then create an SLA

exception request based on the circumstance.

You can search for SLA::Breached issues in the issue tracker using the following label filters:

- Severity 1
- Severity 2
- Severity 3
- Severity 4

Triaging vulnerabilities

We use the Vulnerability Report with filters to focus on items matching our policy and reported on the relevant projects.

1. Analyzers Vulnerability Report

For each item, investigate and either dismiss or confirm it. I

> Refer to Vulnerability status definitions in case you are unsure of what each of them mean.

Triaging vulnerabilities

We use the Vulnerability Report with filters to focus on items matching our policy and reported on the relevant projects.

Dismissing a vulnerability

When there is no doubt a vulnerability is a false-positive, it can be "Dismissed" unless it related to a FedRAMP image (fips).

Select the "Dismiss" option from the vulnerability status options.

Finally, make sure to comment on the vulnerability status change notification to explain why.

Confirming a vulnerability

If the vulnerability impacts a dependency:

1. Evaluate if the dependency (software library, system library, base image, etc.) can be upgraded or removed.

1. Set the vulnerability status to "Confirmed".

1. Release a new version of the analyzer with the dependency upgrade/removal and follow the process on resolving a vulnerability.

1. If the dependency cannot be updated or removed then SLA exception can be requested to extend SLA based on the circumstance.

Resolving a vulnerability

Upon remediating a vulnerability, it will automatically be moved to "Resolved" status by the next scan.

Responsibilities - Support

1. Monitor slack channels for questions, support requests, and alerts. While other team members

may respond to these requests, the engineer assigned to the reaction rotation is expected to handle them primarily.

If a support engineer requests assistance via Slack and it requires investigation or debugging, they should be directed to raise an issue in a dedicated project.

- gast-dynamic-analysis

1. Monitor Section Sec Request For Help project for support requests.

1. Try to resolve issues as much as possible, if you discover that the problem applies to more than one customer - open a public issue and link it. Tag EM/PM.

These items must be triaged continuously throughout the milestone which means they must be checked multiple times a week.

Daily Look Left & Right Before Dev

On any day, each team member should look left and right on our Dynamic Analysis delivery board before they start developing any new work from the workflow::ready for development column, helping to move one issue forward from both directions (2 total). The team follows the workflow states and activities articulated in GitLab's Product Development Flow. Looking left involves grabbing an issue from either the workflow::refinement or the workflow::ready for development columns, whichever has more items, and completing it. Looking right involves grabbing an issue from either the workflow::in review or the workflow::verification columns, whichever has more items, and completing it. After both issues have been moved to a new column, the team member can start ready work from the workflow::ready for development.

The Look Left & Right Before Dev strategy ensures that work items aren't getting stuck on our board, that every team member gets a chance to plan, breakdown, and refine issues, and that code reviews and staging/production verification activities are spread equally across the team. If there is no work to the left, new work can be brought in from our milestone backlog, as defined in our current Milestone Release Planning issue. New work may also be added to the first two columns at any time from sales and support request for help issue, high-priority bugs, and security-related issues.

Blueprinting & Design Docs

When there are too few or no work items in the workflow::refinement or the workflow::ready for development columns, check the current Milestone Release Planning issue for "Work to Blueprint" issues. Assign yourself as a DRI, and assist in scheduling the necessary meetings with the team to review and help blueprint this work for the next milestone. Attempt to create as many child epics and issues as placeholders for eventual planning, breakdown, and refinement. This is a great time for the team to discuss high-level architectural directions, implementation plans, and needs for accomplishing these priorities throughout the next month. All blueprint issues created should receive the following milestone and labels:

text

/milestone {from mapping}

/label ~"section::sec"

/label ~"devops::application security testing"

/label ~"group::dynamic analysis"

/label ~workflow::planning breakdown

If a blueprinting effort is beyond the scope of the Dynamic Analysis group, affecting multiple teams, consider adding it as a design document using this process. A list of all architecture design documents at GitLab can be found [here](#).

Issue breakdown

Team members are empowered to create new issues, attach them to relevant epics, and collaborate with product management to determine if they are included in current scope.

Breakdown principles

Issues are broken down according to the INVEST mnemonic).

- Independent, the issue should have minimal to no dependencies so it can be played in isolation of other issues.
- Negotiable, the issue should be structured such that the PM can prioritize or deprioritize it with minimal effect on the overall feature.
- Valuable, the issue should provide value when released to production.
- Estimable, the issue should make clear what is in scope and out of scope so that engineers can provide an estimate.
- Small, the issue must fit inside one milestone, ideally, design to delivery should take less than two weeks.
- Testable, the issue must be able to be tested. If it can't be tested, it can't be guaranteed to work in future.

Vertically Split

Issues should be split vertically rather than horizontally. Splitting vertically means the whole system will do something noticeably different; splitting horizontally results in trying to realize the fullest possible change in an individual component.

For example, imagine there is an epic to a CRUD feature to a site. Issues could be split up and played in the following order:

1. Read. This is independent, negotiable, estimable, small and testable. It is also valuable because in production an engineer could manually create the data to be read by users.
1. Create. This is independent, estimable, and small. It is testable because the Read functionality has already been built. It is valuable because engineers don't need to manually create data anymore, and it is negotiable because a PM may decide that engineers manually creating data is good enough.
1. Delete. Same as above, and it is valuable because engineers wouldn't need to manually delete records in production.
1. Update. Same as above, and it is often deprioritized by PMs because Create and Delete is often good enough.

Repeated Tasks

There are several maintenance tasks that need to be completed each milestone. Each iteration, an issue is opened and assigned to an engineer on a rotating basis. Those rotating tasks are:

- Review upstream changes, and open an issue to upgrade DAST if the upstream changes provide important improvements
- Review the security dashboard for DAST and address all critical and high issues. Review the dashboards for upstream projects, ZAP and ZAP Extensions

DAST Technologies

The DAST analyzer we build relies heavily on OWASP's ZAP open source software and ZAP Extensions. This means the accuracy and quality of the DAST analyzer is impacted by the quality of the underlying OSS.

We monitor the underlying tools for changes and for vulnerabilities.

ZAPProxy and ZAP Extensions do not have significant test coverage and therefore changes in those tools could impact DAST in unexpected ways. Since our expectation is that customers run DAST in a CI environment and stability and security is of utmost importance, we do not necessarily ship the latest ZAP build. We actively review the ZAP changelog and evaluate whether

new updates deliver value to our customers and their use cases. We may ship a pre-release build of ZAP or a versioned build

if we determine it contains valuable updates for customers and it passes our CI pipelines.

An assigned backend engineer reviews upstream updates at least monthly to identify new bug fixes or features.

Those changes are presented to the Product Manager for prioritization into DAST.

New DAST Technologies

The DAST analyzer is migrating towards using exclusively a browser-based DAST tool that is being built by GitLab. The tool has internally been referred to as Browserker. Browser based DAST was released as GA in 15.7. The browser-based DAST is being delivered iteratively, with each new iteration taking over some parts of analysis previously done by ZAP, with the eventual goal of deprecating ZAP completely.

Fuzzing Technologies

- The API Security product is built using mostly C with some small amounts of Python. Our engineers use Windows VMs for development.
- The coverage guided fuzzing team works primarily in Go, but is also responsible for maintaining the open source fuzzers in JSFuzz, Pythonfuzz, and Javafuzz. The team also maintains the fuzzing examples in which we aim to cover all major programming languages.

Our Approach to Fuzzing

- Fuzzing has one of the largest adoption hurdles of all the AST products. Coverage fuzzing requires customers to write test harnesses, compile applications with special settings and generates highly technical output.

Web API Fuzzing requires generating files that allow the Web API fuzzing tool to know what parts of the application to fuzz.

Our goal is to simplify and reduce as many of the steps that a customer needs to do to get started. We want to focus our efforts on creating samples, defaults, and intelligence that will simplify fuzzing onboarding.

Community Contributions

The Dynamic Analysis welcomes community contributions. Community Contributors-- please make sure to add the label "group::dynamic analysis" to any Merge Requests or Issues to ensure the Dynamic Analysis team sees your contribution.

Community contributions should get prompt feedback from one of the DAST engineers. All engineers on the DAST team are responsible for working with community contributions. If a team member does not have time to review a community contribution, please tag the Engineering Manager, so that they can assign the community contribution to another team member.

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://10az.online.tableau.com//site/gitlab/views/TopEngineeringMetrics16989570521080/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="dynamic analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px"  
src="https://10az.online.tableau.com//site/gitlab/views/MRMetrics/OverallMRs
```